

Bounded Model Checking, Mechanized Invariants, and Conformance Testing for a Price-Time Matching Engine

Ara Aslyan

Strike Technologies, New York, NY, USA
aaslyan@striketechologies.com

Abstract. We report a case study applying three assurance techniques to related artifacts for a feature-rich price-time matching engine. A TLA+ model is exhaustively checked within finite configurations; a Lean 4 executable reference carries machine-checked invariant-preservation proofs; and traces from the model are replayed against a C++ implementation on projected observables. The methods exposed different defects: TLC produced a three-order FIFO counterexample; operational modeling revealed an ambiguity between self-trade decrement and iceberg reload; proof development exposed an insufficient constant bound in the inner matching loop and an MTL/MinQty transcription error; and conformance testing found a C++ disposal defect. Four completed TLC configurations explored 55,439,125 distinct states in aggregate. Lean proves preservation of all modeled book-state invariants for arbitrary finite books satisfying explicit hypotheses, plus unconditional post-only and self-trade-prevention guarantees on emitted trades. A named corollary lifts the result to finite well-formed order sequences from the empty book. These results do not constitute an end-to-end refinement proof of the C++ engine. Raw model-checking logs, theorem sources, tests, and reproduction scripts accompany the artifact.

Keywords: Model checking · Lean 4 · TLA+ · conformance testing · matching engine.

1 Introduction

A continuous limit-order-book matching engine maintains bids and asks and selects trades according to price and arrival priority. Even a compact engine must compose order types, time-in-force rules, partial fills, iceberg reload, post-only handling, self-trade prevention (STP), minimum quantity, stop triggering, and cancellation. Local rules are straightforward; their interactions are not. A triggered stop re-enters the ordinary pipeline, an STP decrement changes quantities without a trade, and a market-to-limit (MTL) order changes type after its first execution.

This paper evaluates a layered workflow on three separately written artifacts: an executable TLA+ state machine, an executable Lean reference, and a C++ implementation. Each layer answers a different question:

- TLC checks every reachable state within explicitly bounded TLA+ configurations and returns traces for violations.
- Lean checks proofs for arbitrary finite data satisfying stated hypotheses, beyond TLC’s size bounds.
- Trace replay tests agreement between projected TLA+ observations and the C++ engine on the traces actually exercised.

The contribution is an applied case study rather than a new matching algorithm or proof technique. Its value lies in the defect witnesses, assurance boundaries, and reproducible cross-artifact checks. The main findings are:

1. TLC found a FIFO violation caused by a triggered stop retaining its original timestamp. The counterexample has been reproduced under a valid three-order configuration and archived with its raw log.
2. Modeling exposed a gap in the interaction of STP DECREMENT and iceberg visibility: decrement can exhaust the visible slice without executing the fill path that reloads it.
3. Lean proof development showed that a constant bound of 100 was insufficient for the inner matching loop and exposed an MTL/MinQty routing error in the executable transcriptions.
4. TLA+-trace replay found a C++ path on which a triggered stop-market remainder could rest on the book.

The artifacts deliberately remain separate. Agreement is tested, not proved, and correspondence between the prose requirements and each formalization remains a validation obligation.

2 Engine and Properties

An order contains an identifier, side, type, quantity, optional limit and stop prices, time in force, optional display and minimum quantities, a post-only flag, and optional STP group and policy. The model supports limit, market, MTL, stop-limit, and stop-market orders; GTC, IOC, FOK, and DAY; iceberg display; and four STP policies. Prices are integers drawn from a finite set in TLC and natural numbers in Lean.

The central state consists of price-indexed bid and ask queues, dormant stops, last trade price, clock, and next identifier. Price levels are ordered best first and queues are ordered by timestamp. Processing performs validation, post-only and FOK/MinQty checks, stop placement or matching, disposal of a remainder, and a bounded stop-trigger cascade.

Table 1 records the canonical property inventory. “Checked” in the TLC column means explicitly listed as an invariant in each checked configuration; it does not generalize beyond the configuration. The Lean column separates the capstone from properties proved at another level.

Table 1. Property coverage. INV-10 is outside both formal models.

ID	Property	TLC	Lean
1	No empty price levels	checked	capstone
2–3	Bid/ask level ordering	construction	AllInv
4	Book uncrossed	checked	AllInv
5	No zero-quantity resting orders	checked	capstone
6	Resting-order status consistency	checked	capstone
7	FIFO within a price level	checked	capstone
8	No resting market orders	checked	capstone
9	Execution at passive price	construction	matching-loop theorem
10	Causal event ordering	not modeled	not modeled
11	No trade from post-only aggressor	checked	unconditional
12	No trade within an STP group	checked	unconditional
13	No resting MTL orders	checked	capstone
14	No resting MinQty marker	checked	capstone

Table 2. Archived TLC results (TLC 2.19, OpenJDK 21, one worker).

Config.	Orders	Qty	Prices	Generated	Distinct	Time
Tiny	2	1	2	2,463,194	1,427,827	0:32
Small	2	2	2	16,920,722	9,230,323	4:12
Medium	2	2	3	49,656,516	29,622,636	14:24
Amend	2	2	2	37,321,458	15,158,339	25:45
Aggregate				106,361,890	55,439,125	

3 Bounded Model Checking in TLA+

The TLA+ artifact represents books as sequences of price-level records and orders as records. `SubmitOrder`, cancellation, amendment, time advance, matching, disposal, and triggered-stop processing form the next-state relation. Order shapes are generated once and filtered by the same well-formedness predicate used by submission. A differential executable check confirms that Lean and TLA+ accept the same 3,830 bounded order shapes.

TLC’s bounds cover submission count, quantity, prices, and logical clock. Terminal states caused by the clock ceiling are expected; runs therefore disable deadlock reporting while retaining invariant checks. Table 2 contains freshly reproduced runs with one worker. The sum of distinct counts measures aggregate exploration workload, not a deduplicated union across configurations.

3.1 FIFO counterexample

Triggered stops originally retained the timestamp assigned when they became dormant. Consider three submissions: (1) a buy limit of quantity one at price one; (2) a sell stop-limit of quantity one with stop and limit price one; and (3) a sell limit of quantity two at price one. Order 3 fills order 1, setting the last trade price and triggering order 2. The remaining unit of order 3 rests first with timestamp 3. The triggered order is appended afterward but retains timestamp 2, yielding the ask queue [3, 2] and violating strict FIFO.

With only the timestamp-refresh line reverted, TLC reproduced this violation under `MAX_ORDERS=3`, `MAX_QTY=2`, and two prices. It found 9,160,154 generated and 4,980,329 distinct states before the violation in 2m41s on the evaluation host. This is a three-order counterexample; no minimality claim is made. The repair assigns the current clock when the stop converts. The raw trace records the seed, fingerprint seed, tool versions, host, and full state sequence.

3.2 Iceberg/STP specification gap

For STP DECREMENT, no trade is emitted; quantities on both orders are reduced. If the resting order is an iceberg and its visible quantity reaches zero while hidden quantity remains, a fill-only reload rule never runs. The result is an unmatchable visible slice of zero. This issue was identified during operational modeling, not by a pre-fix TLC invariant trace. The model reloads such an iceberg, refreshes its timestamp, and moves it to the queue tail. We retain this distinction because a modeled scenario is not equivalent to an archived model-checker counterexample.

4 Mechanized Preservation in Lean 4

The Lean project has no external library dependencies. Its executable definitions mirror the processing phases while favoring proof-oriented data structures. `AllInv` combines uncrossedness with descending bid and ascending ask levels. `BookInvariant` adds no empty levels, no ghosts, status consistency, FIFO, and exclusion of resting market, MTL, and MinQty orders.

The single-step capstone is:

```
theorem process_preserves_BookInvariant
  (b : BookState) (o : Order)
  (hall : AllInv b) (hpok : OrderProcOk o)
  (hsnp : StopsNoPostOnly b) (hb : BookOk b)
  (hstops : StopsWF b) (hok : OrderRestOk o) :
  BookInvariant (process b o).book
```

The hypotheses expose rather than hide the assurance boundary. `BookOk` and `StopsWF` capture structural well-formedness; the order predicates follow from ordinary order well-formedness. Auxiliary theorems show that the book predicates, `AllInv`, and `StopsNoPostOnly` hold initially and are preserved. The named execution-level corollary packages the induction:

```
theorem process_all_preserves_BookInvariant (orders : List Order)
  (hwf : forall o in orders, OrderProcOk o /\ OrderRestOk o) :
  BookInvariant
    (orders.foldl processBook BookState.empty)
```

Thus the theorem covers every finite submitted-order sequence satisfying the per-order predicates, not unconstrained values. Separate unconditional theorems establish post-only and STP safety for the emitted trade list. Passive pricing is proved at the matching-loop level but is not part of the capstone; event ordering is not represented.

4.1 Fuel and proof structure

The original inner matching loop used the constant 100. This made a universal preservation statement false for books with more than 100 matchable opposing orders: fuel could expire while a crossing book remained. The repair computes fuel from the opposing side and proves that every recursive step decreases the relevant order count or incoming quantity. This sufficiency result applies to the inner `doMatch` loop. The outer order/stop-cascade worker likewise uses no fixed limit: `computeProcessFuel` derives its budget from both book sides, all dormant stops, and the incoming order. Thus neither executable processing layer contains an arbitrary constant cutoff. The safety theorems are additionally parametric in the internal worker budget.

The constructive development proves preservation branch by branch. A shorter structural file gives a complementary uncrossedness proof using the matching termination trichotomy, but imports the constructive fuel result and is not an independent assurance argument. The complete reachable-state theorem is in a fourth file. All are imported by the library root, so a default build elaborates them.

The verification gate rejects `sorry`, prints the axiom dependencies of ten top-level theorems, and runs the executable tests. The inspected results depend only on `propext`, `Classical.choice`, and `Quot.sound` as applicable; tactics such as `omega` produce terms checked by the Lean kernel.

5 Trace-Based C++ Conformance

The C++ engine is separately implemented with STL containers. Conformance is tested by generating short TLA+ target-search traces, converting submissions and expected observations to JSON, replaying the submissions, and comparing top of book, order and stop counts, last trade price, and fills after each step. These are projected observations, not full-state equivalence.

A fresh fast sweep ran 117 target searches: 13 seeded scenarios by nine structural targets. Ninety-four convertible traces were replayed, comprising 109 observation steps, 384 submitted orders, and 40 fills; all replayed traces passed with no divergence. Some target searches terminate without the named target but still yield a syntactically convertible TLC behavior, which is why the number of converted traces can exceed the 57 runs that reached their target. Twenty-three runs yielded no convertible behavior. The safety phase used a deliberately short 120-second cap: four scenarios completed and nine timed out without a violation in the explored prefix. These timeouts are not reported as passes.

The fast sweep is evaluation evidence, while the verification gate contains a smaller deterministic regression set (10 traces, 21 steps) plus 65 directed C++ tests and a 2,000-step shadow differential. The historical C++ defect was a disposal path allowing a persistent triggered stop-market remainder to rest. Trace replay localized the mismatch; after repair, market remainders are canceled rather than inserted.

Conformance cannot establish correctness for untested paths, and two implementations can agree on the same error. A specification-rooted oracle is useful precisely when it differs from the implementation’s design lineage, but it still supplies testing rather than refinement.

6 Cross-Layer Lessons and Limitations

The layers found different faults because their quantifiers differ. TLC explores complete bounded executions and produces witnesses. Lean quantifies over arbitrary finite structures satisfying hypotheses, exposing bounds that small configurations never reach. Conformance testing addresses transcription and implementation gaps but only for sampled traces. No one result subsumes the others.

The artifact has several explicit limits. TLC prices, quantities, submissions, and time are finite; only one non-null STP group identifier is used; GTD timing is simplified; and event ordering is absent. The Lean theorem is about the Lean executable and requires well-formed inputs and state. There is no Lean-to-C++ or TLA+-to-C++ refinement theorem. The prose-to-formal correspondence is reviewed and differentially tested but not proved. The MTL/MinQty episode itself demonstrates why this distinction matters: a passing precheck could take an erroneous disposal route in both formal executable transcriptions until the full invariant proof forced that case to be resolved.

Threats to validity include the small TLC domains, model and implementation sharing terminology, incomplete long-running conformance safety searches, and hardware-dependent timing. We therefore archive raw counts and distinguish a completed exploration, a violation-terminated run, a timeout, and a replay pass. Aggregate distinct-state counts are workload totals rather than unique states across models.

7 Related Work

Industrial TLA+ case studies demonstrate the value of executable specifications and counterexample traces [7,6]; our focus is their composition with a mechanized executable and implementation replay. Model-based testing provides the broader basis for deriving tests from transition systems [11,12].

Mechanized financial-market work includes verified trades and double-auction formalizations [10,5,2], while automated reasoning has been applied directly to financial algorithms [8,9]. Our contribution differs in emphasizing feature interaction across three non-refined artifacts rather than a new auction theorem. The Lean termination argument follows established proof-assistant treatments of recursive definitions and well-founded measures [3,1].

Differential testing detects behavioral disagreement but cannot identify the correct side or errors shared by both implementations [4]. Here the TLA+ projection supplies a third oracle, while the Lean proof supplies unbounded invariant reasoning for a related executable.

8 Reproducibility and Conclusion

The artifact provides a fast gate and a deep gate. The fast gate builds Lean, checks proof closure and axiom footprints, runs Lean and C++ tests, TLC smoke and invariant-coverage checks, conformance replay, shadow differential, and the Lean/TLA+ well-formedness differential. The deep gate reruns the four completed TLC configurations; a bounded three-order attempt is reported as a skip unless it completes. Raw logs include commands, hashes, versions, seeds, queue sizes, and host details. CI runs the fast gate and paper build on changes and schedules the deep gate.

This case study supports a deliberately delimited conclusion: bounded TLA+ exploration, Lean invariant preservation, and model-based C++ conformance testing expose complementary defect classes. Lean proves the modeled book-state properties along finite well-formed executions of its reference; TLC exhausts only stated finite configurations; and C++ agreement is tested on replayed projections. Together they provide stronger evidence than any layer alone, but not an end-to-end proof of the deployed implementation.

Data availability. Source, proof artifacts, configurations, raw logs, and scripts are available in the [project repository](#) under the MIT license. The repository records SHA-256 hashes for archived model-checking inputs. A versioned archival DOI should replace this repository URL in the camera-ready version once the release is deposited.

AI assistance. The formal artifacts and manuscript were developed interactively with a large language model used as a coding and drafting assistant. Proof terms are accepted by the Lean kernel, and empirical claims are tied to archived tool outputs; responsibility for the claims and artifact remains with the author.

References

1. Bove, A., Krauss, A., Sozeau, M.: Partiality and recursion in interactive theorem provers—an overview. *Math. Struct. Comput. Sci.* 26(1), 38–88 (2016)
2. Garg, M., Natarajan, R., Sarswat, S., Singh, A.K.: Double auctions: formalization and automated checkers. *J. Autom. Reason.* 69, 17 (2025)
3. Krauss, A.: Automating Recursive Definitions and Termination Proofs in Higher-Order Logic. Ph.D. thesis, Technische Universität München (2009)
4. McKeeman, W.M.: Differential testing for software. *Digital Technical Journal* 10(1), 100–107 (1998)
5. Natarajan, R., Sarswat, S., Singh, A.K.: Verified double sided auctions for financial markets. In: *ITP 2021. LIPIcs*, vol. 193, pp. 28:1–28:18 (2021)
6. Newcombe, C.: Why Amazon chose TLA+. In: *ABZ 2014. LNCS*, vol. 8477, pp. 25–39. Springer (2014)
7. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M.: How Amazon Web Services uses formal methods. *Commun. ACM* 58(4), 66–73 (2015)

8. Passmore, G.O., Ignatovich, D.: Formal verification of financial algorithms. In: CADE 2017. LNCS, vol. 10395, pp. 26–41. Springer (2017)
9. Passmore, G.O.: Some lessons learned in the industrialization of formal methods for financial algorithms. In: FM 2021. LNCS, vol. 13047, pp. 717–721. Springer (2021)
10. Sarswat, S., Singh, A.K.: Formally verified trades in financial markets. In: ICFEM 2020. LNCS, vol. 12531, pp. 217–232. Springer (2020)
11. Tretmans, J.: Model based testing with labelled transition systems. In: Formal Methods and Testing. LNCS, vol. 4949, pp. 1–38. Springer (2008)
12. Utting, M., Pretschner, A., Legeard, B.: A taxonomy of model-based testing approaches. *Softw. Test. Verif. Reliab.* 22(5), 297–312 (2012)